# Nearest neighbors, matrix product, matrix factorization

Lecture notes. AI in Industry, ETH Zurich, weekend 2, Friday 13 November 2026, 08:15 to 12:00.

## 1 The inner product

**Rating.** A number from $-2$ to 2: 2 loved, 1 liked, 0 neutral, $-1$ disliked, $-2$ hated. A rating of 1 to 5 stars becomes the number of stars minus 3.

**Customer.** A row of ratings, one per film.

**Inner product.** Of two rows of the same length, $a = (a_1, \ldots, a_n)$ and $b = (b_1, \ldots, b_n)$: the sum of the products of their entries,

$$a \cdot b = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n.$$

**One film.** The product of two customers' ratings says how much they agree: positive when both are on the same side, negative when they are on opposite sides, 0 when one of them is neutral, further from 0 when the opinions are stronger. Jaws: Ivo and Pia, $2 \times 2 = 4$; Ivo and Rosa, $2 \times (-2) = -4$; Pia and Kai, $2 \times 1 = 2$; Kai and Tom, $1 \times (-1) = -1$.

**Several films.** The inner product adds the agreements of all films. Vera and Kai:

| | Jaws | | The Shining | | Notting Hill | | |
|---|---|---|---|---|---|---|---|
| Vera | 2 | | 1 | | $-1$ | | |
| Kai | 1 | | 2 | | $-2$ | | |
| | 2 | + | 2 | + | 2 | = | 6 |

With ratings 1 and $-1$ only, the inner product is the number of matches minus the number of mismatches.

Game: Once Upon a Video Store, acts 1 to 3.

## 2 The nearest neighbor classifier

**Classifier.** A rule that predicts the class of a new case. Here the case is a newcomer, and the class is her rating of a film she has not seen.

**Nearest neighbor classifier.**

1. Similarity: the inner product of the newcomer's row with each regular customer's row, on the films the newcomer rated.
2. Nearest neighbor: the regular with the largest inner product.
3. Prediction: the nearest neighbor's rating of the film.

$k$ **nearest neighbors.** The $k$ regulars with the largest inner products vote with their ratings of the film. The prediction is the rating with the most votes; on a tie, the nearer neighbor's vote wins.

Example. Vera rated three films. The regulars, sorted by their inner product with her:

| | Jaws | The Shining | Notting Hill | Love Actually | inner product with Vera |
|---|---|---|---|---|---|
| Vera | 2 | 1 | −1 | ? | |
| Kai | 1 | 2 | −2 | −2 | 6 |
| Pia | 2 | 1 | 2 | 2 | 3 |
| Ivo | 2 | −2 | 0 | 1 | 2 |
| Jana | 0 | 1 | 0 | 0 | 1 |
| Tom | −1 | −2 | −1 | −1 | −3 |
| Rosa | −2 | −1 | 2 | 2 | −7 |

Nearest neighbor: Kai, 6. Prediction for Love Actually: −2, Kai's rating. Vera's rating after watching it: −2.

A count of exact matches picks Pia (two films) over Kai (none); Pia's rating of Love Actually is 2. The inner product weighs each agreement by the strength of both ratings.

Game: Once Upon a Video Store, act 4. Notebook: nearest neighbors.

## 3 The matrix product

**Matrix.** A table of numbers. Its size is $n \times m$: $n$ rows, $m$ columns.

**Profiles.** Each customer answers three questions with a number from −2 to 2: real story (2) or fiction (−2), comedy (2) or drama (−2), action (2) or calm (−2). A film critic answers the same questions for every film. $P$ holds the customers' answers, one row per customer. $Q$ holds the critic's answers, one column per film.

**Embedding.** The numbers that describe a customer (a row of $P$) or a film (a column of $Q$).

**Score.** The inner product of a customer's row and a film's column. The higher the score, the better the film suits the customer.

**Matrix product.** $P$ of size $n \times k$ times $Q$ of size $k \times m$, defined when $P$ has as many columns as $Q$ has rows. $PQ$ has size $n \times m$. Its entry in row $i$ and column $j$ is the inner product of row $i$ of $P$ and column $j$ of $Q$. With $Q$ drawn above $PQ$ and $P$ on its left, each entry sits where its row and its column meet:

| | The Intouchables | Apollo 13 | Men in Black | Interstellar |
|---|---|---|---|---|
| real | 2 | 2 | −2 | −2 |
| comedy | 1 | −2 | 1 | −2 |
| action | −1 | 1 | 2 | 0 |

$Q$

| | real | comedy | action |
|---|---|---|---|
| Sara | 2 | 1 | −2 |
| Leo | −2 | 2 | 2 |
| Maya | 1 | −2 | 1 |

$P$

| | | | |
|---|---|---|---|
| 7 | 0 | −7 | −6 |
| −4 | −6 | 10 | 0 |
| −1 | 7 | −2 | 2 |

$PQ$

Sara and The Intouchables: $2 \times 2 + 1 \times 1 + (-2) \times (-1) = 4 + 1 + 2 = 7$. Each customer's highest score: Sara, The Intouchables (7); Leo, Men in Black (10); Maya, Apollo 13 (7).

Sizes: $5 \times 4$ times $4 \times 6$ gives $5 \times 6$. The product of $2 \times 3$ and $4 \times 2$ is undefined: 3 columns against 4 rows.

Game: Once Upon a Matchmaker. Notebook: matrix factorization, part 1.

# 4 Matrix factorization

## 4.1 The problem

**Matrix factorization.** Given only the ratings $R$, $n$ customers by $m$ films, find $P$ ($n \times k$) and $Q$ ($k \times m$) whose product $PQ$ is close to $R$. The number $k$ is chosen in advance. The rows of $P$ and the columns of $Q$ are embeddings; their numbers have no names.

**Prediction.** The entry of $PQ$ for a customer and a film.

**Loss.** Of one rating: $(\text{prediction} - \text{rating})^2$.

**Total loss.** The sum of the losses of the known ratings. Matrix factorization looks for the $P$ and $Q$ with the smallest total loss.

Example: three customers, two films, $k = 2$, a first guess of $P$ and $Q$:

| | 1 | 2 |
|---|---|---|
| Ida | −1 | −2 |
| Raj | −2 | 0 |
| Elsa | 2 | 2 |

$P$

| | Paddington | The Martian |
|---|---|---|
| 1 | −1 | −2 |
| 2 | 2 | −1 |

$Q$

| | | |
|---|---|---|
| | −3 | 4 |
| | 2 | 4 |
| | 2 | −6 |

predictions $PQ$

| | | |
|---|---|---|
| | 2 | 1 |
| | −1 | 2 |
| | 1 | −2 |

ratings $R$

| | | |
|---|---|---|
| | 25 | 9 |
| | 9 | 4 |
| | 1 | 16 |

losses

Total loss: $25 + 9 + 9 + 4 + 1 + 16 = 64$.

## 4.2 One number at a time

For one number $w$ of $P$ or $Q$:

1. Before: the total loss.
2. After: the total loss with $w$ changed to $w + 0.01$.
3. Slope: $\nabla = (\text{after} - \text{before})/0.01$.
4. Update: $w \leftarrow w - \alpha\nabla$, with the step size $\alpha = 0.02$.

A negative slope moves $w$ up, a positive slope moves it down. A round updates every number once, the numbers of $P$ first. Rounds repeat until the total loss is below 0.001.

Example: The Martian's second number, $w = -1$. Before: 64.00. After, with $w = -0.99$: 63.72. Slope: $\nabla = -0.28/0.01 = -28$. Update: $w \leftarrow -1 - 0.02 \times (-28) = -0.44$; the total loss falls to 50.86. After 77 rounds the total loss is below 0.001 and every prediction is within 0.03 of its rating.

Game: Once Upon a Streaming Service. Notebook: matrix factorization, part 2.

### 4.3 Gradient descent

**Slope rule.** For a number $w$ and a known rating whose prediction contains $w$: $2 \times$ partner $\times$ (prediction $-$ rating), where the partner is the number that $w$ multiplies in that prediction. A rating whose prediction does not contain $w$ gives 0.

**Slope of the total loss.** The sum of the slope rule over the known ratings. The slope measured with a change of 0.01 is close to it.

**Gradient.** The slopes of all numbers of $P$ and $Q$.

**Gradient descent.** Every number moves at once, $w \leftarrow w - \alpha \times$ slope, again and again until the total loss stops falling.

Example: from the first guess above, with $\alpha = 0.02$, one step takes the total loss from 64 to 19.68, and 118 steps take it below 0.0001.

Game: Once Upon a Streaming Service, optional part. Notebook: matrix factorization, optional, parts 3 and 4.

### 4.4 Predicting unknown ratings

The total loss uses the known ratings only. After the descent, $PQ$ has an entry for every customer and every film; for an unknown rating, that entry is the prediction.

Example: real ratings from MovieLens[1], 10 users and 12 films, each rating the number of stars minus 3, three ratings hidden. Gradient descent with $k = 2$, $\alpha = 3/117$ and 200 steps takes the total loss of the 117 known ratings from 215.17 to 29.35. The hidden ratings and their predictions:

| Film | MovieLens user | Prediction | Rating |
|---|---|---|---|
| Independence Day | 64 | 0.67 | 0 |
| Braveheart | 28 | 0.62 | 0.5 |
| Seven | 239 | 1.98 | 2 |

Game: Once Upon a Streaming Service, the whole picture.

## 5 Games and notebooks

Games, on the course site:

| | |
|---|---|
| Once Upon a Video Store | https://eth-ainit-hs26.github.io/AINIT-PAGE/viz/we2/video-store/ |
| Once Upon a Matchmaker | https://eth-ainit-hs26.github.io/AINIT-PAGE/viz/we2/matchmaker-part-1/ |
| Once Upon a Streaming Service | https://eth-ainit-hs26.github.io/AINIT-PAGE/viz/we2/matchmaker-part-2/ |
| Streaming Service, optional part | https://eth-ainit-hs26.github.io/AINIT-PAGE/viz/we2/matchmaker-optional/ |

Notebooks, in the public repository https://github.com/eth-ainit-hs26/w2-cx-public. Each link opens a notebook in Colab:

| | |
|---|---|
| Nearest neighbors | student, solutions |
| Matrix factorization, parts 1 and 2 | student, solutions |
| Matrix factorization, optional parts 3 and 4 | student, solutions |

[1] MovieLens ml-latest-small, GroupLens Research, University of Minnesota. F. Maxwell Harper and Joseph A. Konstan, The MovieLens Datasets: History and Context, ACM Transactions on Interactive Intelligent Systems 5(4), 2015.